

Numerically Stable Dynamic Bicycle Model for Discrete-time Control

Qiang Ge¹, Qi Sun¹, Shengbo Eben Li¹, Sifa Zheng^{1*}, Wei Wu² and Xi Chen²

Abstract—Dynamic/Kinematic model is of great significance in decision and control of intelligent vehicles. However, due to the singularity of dynamic models at low speed, kinematic models have been the only choice under such driving scenarios. Inspired by the concept of backward Euler method, this paper presents a discrete dynamic bicycle model feasible at any low speed. We further give a sufficient condition, based on which the numerical stability is proved. Simulation verifies that (1) the proposed model is numerically stable while the forward-Euler discretized dynamic model diverges; (2) the model reduces forecast error by up to 65% compared to the kinematic model. As far as we know, it is the first time that a dynamic bicycle model is qualified for urban driving scenarios involving stop-and-go tasks.

I. INTRODUCTION

Vehicle models are widely used in sensing, decision-making and control of intelligent vehicles - preferably those that meet the following requirements: differentiability, non-singularity, numerical stability. Specifically, such a model describes how the state evolves in a Kalman filter [1], simulates kinematically-feasible trajectories of ego vehicle [2] or surrounding vehicles, and forecasts future states in a Model Predictive Controller (MPC) [3]. Moreover, model-based reinforcement learning, such as Approximate Dynamic Programming (ADP), also relies on a state transition model [4].

The modeling of a typical front-steered four-wheel ground vehicle has been developed at various scales, varying from 1-Degree-of-Freedom (DoF) integrator [6] to 15-DoF full car. They can be divided into two categories, i.e., kinematic and dynamic model [5]. While the kinematic model is deduced via geometrical relations, the dynamic model stems from force analysis. Generally, a decision algorithm from upstream calculates two variables, namely acceleration and front-wheel steer angle, to manipulate the vehicle in the $x - y$ plane. In this case, the vehicle has 3 kinds of motion: translation in global x coordinate, translation in global y coordinate, and rotation around the vertical axis. Another common practice is to consider the vehicle as an equivalent bicycle, given it is bilaterally symmetric and the effects of lateral payload shift on the tire lateral behavior are small. The dynamic bicycle model corresponding to the aforementioned motions has 3

DoFs, whereas its kinematic counterpart has only 2 DoFs as there is no lateral slip.

The two types of bicycle models are applied in different scenarios or tasks. Kinematic models are adopted at low speeds, especially in stop-and-go scenarios, whereas dynamic models can only serve higher speeds. The former is believed to be irreplaceable because all dynamic models encounter a singularity point when approaching zero speed, since the tire slip angle estimation term has the vehicle velocity in the denominator [7]. As a result, a kinematic model is almost the default approach for low-speed tasks like autonomous parking [8], autonomous intersection management [9] and multi-vehicle formation [10]. However, the modeling error will inevitably grow with driving speed, as tire sideslip angle is no longer negligible.

An insight into the singularity problem requires us to transform the continuous differential kinematic/dynamic bicycle model into a discrete version, i.e. difference equations. Initial Value Problem (IVP) of Ordinary Differential Equations (ODE) is handled with many mature mathematical techniques, like the Euler method, Runge-Kutta methods, and linear multistep methods [11]. With forward Euler method, the next-step state can be expressed as an explicit linear combination of the state derivative at current step, the current state, and the step length, which accommodates any form of derivative functions well. However, forward Euler method can have poor numerical stability, which means the modeling error diverges with prediction step increases. Backward Euler method is an implicit method, which has usually better numerical stability but poor computational efficiency. The next-step state has to be approximated by iteration. More importantly, in learning-based applications like ADP, the model should have an explicit form of forward propagation so as to build a computational graph. Otherwise, the derivative of next-step state to current state, which is part of the loss function's gradient, cannot be automatically calculated by machine learning libraries [12]. Thus, currently, the application of backward Euler method is mainly limited to open-loop simulation instead of closed-loop optimization.

Recently, neural network-based model has become remarkable progress in this field [13]. Nevertheless, as a new neural model has to be trained for each different vehicle, formula-based models would be more generic thanks to their interpretable parameters.

Therefore, the discrete vehicle model for planning/control in the two-dimensional plane is either unstable or not accurate enough. This paper tries to derive a more accurate yet numerically stable state transition model, with a tailored discretization method. The rest content is organized as fol-

*This work is supported by National Key R&D Program of China with 2018YFB1600600. This work is also supported by National Key R&D Program of China with 2018YFE0192900, and Geekplus Technology Co., Ltd. with 20202001297. All correspondence should be sent to S. Zheng.

¹Qiang Ge, Qi Sun, Shengbo Eben Li and Sifa Zheng are with the School of Vehicle and Mobility, Tsinghua University, Beijing, China. (gq17@mails.tsinghua.edu.cn, qisun@mail.tsinghua.edu.cn, {lishbo, zsf}@tsinghua.edu.cn)

²Wei Wu and Xi Chen are with the Geekplus Technology Co., Ltd., Beijing, 100020, China. ({merlinwu, chenxi}@geekplus.com)

lows. In Section II, both kinematic and dynamic single-track models are illustrated. Section III derives an explicit discrete dynamic bicycle model with due simplifications, which is inspired by the idea of backward Euler method. Further, we manage to give a sufficient condition of provable numerical stability in section IV. The accuracy, stability, computational efficiency as well as support to stop-and-go tasks, are verified in Section V. Section VI gives some concluding remarks.

II. BICYCLE MODEL

A. Dynamic Model

The dynamic bicycle model is given in (1). The six state variables are respectively horizontal position x , vertical position y , yaw angle φ , longitudinal velocity u , lateral velocity v and yawrate ω . The input is front wheel steering angle δ and acceleration command a . Here $a = \frac{T_w}{mR_w}$, where T_w is the longitudinal drive torque, m is the vehicle mass, R_w is rolling radius of the drive wheel. The rolling resistance and air drag are ignored.

$$\dot{X} = f(X, U) = \begin{bmatrix} u \cos \varphi - v \sin \varphi \\ u \sin \varphi + v \cos \varphi \\ \omega \\ a + v\omega - \frac{1}{m}F_{Y1} \sin \delta \\ -u\omega + \frac{1}{m}(F_{Y1} \cos \delta + F_{Y2}) \\ \frac{1}{l_z}(l_f F_{Y1} \cos \delta - l_r F_{Y2}) \end{bmatrix} \quad (1)$$

where

$$X = \begin{bmatrix} x \\ y \\ \varphi \\ u \\ v \\ \omega \end{bmatrix}, \quad U = \begin{bmatrix} a \\ \delta \end{bmatrix} \quad (2)$$

The lateral forces F_{Y1} , F_{Y2} are functions of sideslip angle

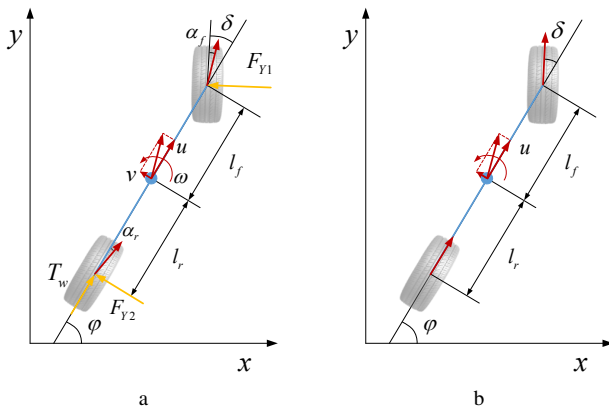


Fig. 1. Bicycle model
(a) dynamic, (b) kinematic

α_f and α_r . Since the extreme case that the vehicle has big lateral acceleration is not under consideration, it is reasonable

to assume that

$$F_{Y1} = k_f \alpha_1 \approx k_f \left(\frac{v + l_f \omega}{u} - \delta \right) \quad (3a)$$

$$F_{Y2} = k_r \alpha_2 \approx k_r \frac{v - l_r \omega}{u} \quad (3b)$$

It should be emphasized that the estimation of α_f and α_r by (3) is necessary, no matter which tire force model is adopted. Therefore, the near-zero singularity introduced by u in the denominator is inevitable.

In addition, we assume $\cos \delta \approx 1$ for concision. One may also retain $\cos \delta$ in the following steps and derive a more accurate discrete model. The accuracy difference will be discussed in section V.

B. Kinematic Model

The widely used kinematic bicycle model assumes that both front and rear wheels have only longitudinal rolling movement, but no lateral slip. This is less accurate because tire characteristics are neglected.

$$\dot{X}_{\text{kine}} = f_{\text{kine}}(X_{\text{kine}}, U) = \begin{bmatrix} u \cos \varphi - \frac{l_r}{l_f + l_r} u \tan \delta \sin \varphi \\ u \sin \varphi + \frac{l_r}{l_f + l_r} u \tan \delta \cos \varphi \\ a \\ \frac{1}{l_f + l_r} u \tan \delta \end{bmatrix} \quad (4)$$

where

$$X_{\text{kine}} = \begin{bmatrix} x \\ y \\ u \\ \varphi \end{bmatrix} \quad (5)$$

Note that four state variables are enough here because the kinematic model has only 2 DoFs. Besides, the acceleration a of (4) is defined as the derivative of u in the local (vehicle-attached) coordinate system. a of (1) is the absolute longitudinal acceleration vector component projected onto the local reference frame, which could be measured by an Inertial Measurement Unit (IMU).

III. DISCRETIZATION

To apply the ordinary differential equations (1) to controller modeling or trajectory planning, the foremost step is to solve the Initial Value Problem (IVP) with numerical methods.

As a typical implicit method, the main defect of backward Euler method is that X_{k+1} has to be found by solving the root-finding problem (6), usually using fixed-point iteration. T_s is the time steplength.

$$X_{k+1} - T_s f(X_{k+1}, U_k) = X_k \quad (6)$$

In order to retain the advantage of explicit methods, we derive X_{k+1} in a variable-by-variable fashion. x_{k+1} , y_{k+1} and φ_{k+1} are calculated by forward Euler method. The derivatives of state variable x , y , φ do not contain themselves, which makes backward Euler method unnecessary. u_{k+1} is also

derived by forward Euler method after simplifying $v\omega - \frac{1}{m}F_{Y1}\sin\delta$ into zero. This can be regarded as neglecting the longitudinal resistance force ($-\frac{1}{m}F_{Y1}\sin\delta \rightarrow 0$) first, and then substituting body-fixed coordinate for space-fixed coordinate system ($a \rightarrow \dot{u}$ instead of $a + v\omega \rightarrow \dot{u}$).

After that, v_{k+1} and ω_{k+1} are solved from

$$\begin{aligned} v_{k+1} - T_s \left(-u_k \omega_k + \frac{1}{m} (F_{Y1}(u_k, v_{k+1}, \omega_k, \delta_k) + F_{Y2}(u_k, v_{k+1}, \omega_k)) \right) &= v_k \\ \omega_{k+1} - T_s \left(\frac{1}{I_z} (l_f F_{Y1}(u_k, v_k, \omega_{k+1}, \delta_k) - l_r F_{Y2}(u_k, v_k, \omega_{k+1})) \right) &= \omega_k \end{aligned}$$

Note that all $k+1$ -step variables but the unsolved v_{k+1} , ω_{k+1} are approximated by k -step states respectively, compared to the standard backward Euler formulation. Therefore, owing to the linear sideslip force, the above formulas have analytical solutions.

Technically the discretization is neither a standard forward nor backward Euler method, but a variant inspired by them. It is the adoption of backward conception that makes the model stable, which is proved in the next section. The simplification and approximation are also necessary for numerical stability. The resultant modeling error will be evaluated in Section V. We propose the following discrete model

$$X_{k+1} \triangleq F(X_k, U_k) \quad (7)$$

which is extended as

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \varphi_{k+1} \\ u_{k+1} \\ v_{k+1} \\ \omega_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + T_s (u_k \cos \varphi_k - v_k \sin \varphi_k) \\ y_k + T_s (v_k \cos \varphi_k + u_k \sin \varphi_k) \\ \varphi_k + T_s \omega_k \\ u_k + T_s a_k \\ \frac{mu_k v_k + T_s (l_f k_f - l_r k_r) \omega_k - T_s k_f \delta_k u_k - T_s mu_k^2 \omega_k}{mu_k - T_s (k_f + k_r)} \\ \frac{I_z u_k \omega_k + T_s (l_f k_f - l_r k_r) v_k - T_s l_f k_f \delta_k u_k}{I_z u_k - T_s (l_f^2 k_f + l_r^2 k_r)} \end{bmatrix} \quad (8)$$

IV. NUMERICAL STABILITY

A. Linearization of Difference Equation

The nonlinear recursive difference equation (7) can be seen as a reference point of the following nonlinear function

$$Y = F(X, U) \quad (9)$$

Using *Taylor's theorem*,

$$Y - X_{k+1} \approx \left. \frac{\partial F}{\partial X^T} \right|_{\substack{X=X_k \\ U=U_k}} (X - X_k) + \left. \frac{\partial F}{\partial U^T} \right|_{\substack{X=X_k \\ U=U_k}} (U - U_k) \quad (10)$$

denote $X - X_k$ as $\vec{\varepsilon}_k$, and $Y - X_{k+1}$ as $\vec{\varepsilon}_{k+1}$, after approximation,

$$\vec{\varepsilon}_{k+1} = J_{F,X}(X_k, U_k) \vec{\varepsilon}_k \triangleq A_k \vec{\varepsilon}_k \quad (11)$$

Note that from (10) to (11), the difference item of input U was ignored. This is because the input U is defined rather than estimated by us. In another word, it is reasonable not to consider any error originating from $U - U_k$.

Additionally, note that the first three state variables, i.e. x_k , y_k and φ_k are merely combination and integral of the other variables u_k , v_k and ω_k . Thus, we can alternatively consider a

dynamic system (still termed $F(X_k, U_k)$ for simplicity) with X_k containing only u_k, v_k and ω_k , which distinguishes (8) from other systems.

According to the definition of Jacobian matrix,

$$\begin{aligned} A_k &= \begin{bmatrix} \frac{\partial F}{\partial u_k} & \frac{\partial F}{\partial v_k} & \frac{\partial F}{\partial \omega_k} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ b_{1,k} & \frac{mu_k - T_s(k_f + k_r)}{T_s(l_f k_f - l_r k_r)} & \frac{T_s(l_f k_f - l_r k_r) - T_s mu_k^2}{mu_k - T_s(k_f + k_r)} \\ b_{2,k} & \frac{I_z u_k - T_s(l_f^2 k_f + l_r^2 k_r)}{I_z u_k - T_s(l_f k_f - l_r k_r)} & \frac{I_z u_k}{I_z u_k - T_s(l_f^2 k_f + l_r^2 k_r)} \end{bmatrix} \\ &\triangleq \begin{bmatrix} 1 & \vec{0} \\ \vec{b}_k & \hat{A}_k \end{bmatrix} \end{aligned} \quad (12)$$

B. Analysis of Numerical Stability

Our definition of numerical stability is given hereby.

Definition 1: The system described by (8) is called numerically stable if such boundedness characteristic is satisfied:

$$\forall \vec{\varepsilon}_0 \in \mathcal{R}^{3 \times 1}, \exists C \in \mathcal{R}, s.t. \lim_{k \rightarrow \infty} \|A_k \cdots A_1 A_0 \vec{\varepsilon}_0\| \leq C$$

As a matter of fact, if (11) can be seen as a switched discrete linear system, the concept of *Joint Spectral Radius* could be of help in the stability proof [14]. However, since the matrix A_k in this case cannot be included by a finite set, we have to seek other methods.

Definition 1 requires that, each entry of the long product $A_k \cdots A_1 A_0$ does not grow infinity with increasing k . This requirement, however, is difficult to be satisfied as the coefficient matrix A_k is varying with each step k . More specifically, we believe that rigorous stability only holds under certain conditions. Therefore, in what follows we manage to give a sufficient condition that will lead to this good feature.

As we have

$$\begin{aligned} A_k \cdots A_1 A_0 &= \begin{bmatrix} 1 & \vec{0} \\ \vec{b}_k & \hat{A}_k \end{bmatrix} \cdots \begin{bmatrix} 1 & \vec{0} \\ \vec{b}_0 & \hat{A}_0 \end{bmatrix} \\ &\triangleq \begin{bmatrix} 1 & \vec{0} \\ \vec{b}_{k,0} & \hat{A}_{k,0} \end{bmatrix} \end{aligned} \quad (15)$$

The numerical stability will be guaranteed as long as $\hat{A}_{k,0}$ and $\vec{b}_{k,0}$ are both bounded. The sufficient condition is given as follows.

Condition 1: $\|\hat{A}_k\| \leq 1, k = 0, 1, 2, \dots$

If Condition 1 is satisfied, the long product $\hat{A}_{k,0}$ will decay exponentially. Moreover, we notice that $\hat{A}_k$ involves u_k and step length T_s , which means that the norm of $\hat{A}_k$ will not be an arbitrary value but restricted in a certain range. This physical meaning could be crucial for the establishment of Condition 1.

$$b_{1,k} = \frac{(mv_k - T_s k_f \delta_k - 2T_s mu_k \omega_k) [mu_k - T_s (k_f + k_r)] - m [mu_k v_k + T_s (l_f k_f - l_r k_r) \omega_k - T_s k_f \delta_k u_k - T_s mu_k^2 \omega_k]}{[mu_k - T_s (k_f + k_r)]^2} \quad (13)$$

$$b_{2,k} = \frac{(I_z \omega_k - T_s l_f k_f \delta_k) [I_z u_k - T_s (l_f^2 k_f + l_r^2 k_r)] - I_z [I_z u_k \omega_k + T_s (l_f k_f - l_r k_r) v_k - T_s l_f k_f \delta_k u_k]}{[I_z u_k - T_s (l_f^2 k_f + l_r^2 k_r)]^2} \quad (14)$$

1) *boundedness of $\hat{A}_{k,0}$:*

Proposition 1: $\hat{A}_{k,0}$ is always bounded when k approaches infinity, given that Condition 1 is satisfied.

Proof

$$\begin{aligned} \|\hat{A}_{k,0}\| &= \|\hat{A}_k \cdots \hat{A}_1 \hat{A}_0\| \\ &\leq \|\hat{A}_k\| \cdots \|\hat{A}_1\| \|\hat{A}_0\| \\ &\leq 1 \end{aligned} \quad (16)$$

Remark 1: Given that Condition 1 is established, with the *submultiplicative* property of induced norm, we prove that $\|\hat{A}_{k,0}\|$ is always bounded by 1 as k increases. Moreover, as long as $\|\hat{A}_k\| \neq 1$, each entry of the submatrix $\hat{A}_{k,0}$ in (15) will converge to zero exponentially, which is even more favorable for stability.

2) *boundedness of $\vec{b}_{k,0}$:*

Proposition 2: $\vec{b}_{k,0}$ is always bounded when k approaches infinity, given that Condition 1 is satisfied.

Proof

$$\begin{aligned} \vec{b}_{k,0} &= \vec{b}_k + \hat{A}_k \vec{b}_{k-1,0} \\ &= \vec{b}_k + \hat{A}_k \vec{b}_{k-1} + \hat{A}_k \hat{A}_{k-1} \vec{b}_{k-2,0} \\ &= \cdots \\ &= \vec{b}_k + \hat{A}_k \vec{b}_{k-1} + \cdots + \hat{A}_k \hat{A}_{k-1} \cdots \hat{A}_1 \vec{b}_0 \end{aligned} \quad (17)$$

$$\begin{aligned} \|\vec{b}_{k,0}\| &= \|\vec{b}_k + \hat{A}_k \vec{b}_{k-1} + \cdots + \hat{A}_k \hat{A}_{k-1} \cdots \hat{A}_1 \vec{b}_0\| \\ &\leq \|\vec{b}_k\| + \|\hat{A}_k \vec{b}_{k-1}\| + \cdots + \|\hat{A}_k \hat{A}_{k-1} \cdots \hat{A}_1 \vec{b}_0\| \\ &\leq \|\vec{b}_k\| + \|\hat{A}_k\| \|\vec{b}_{k-1}\| + \cdots + \|\hat{A}_k\| \cdots \|\hat{A}_1\| \|\vec{b}_0\| \\ &\leq \|\vec{b}_*\| + \|\hat{A}_*\| \|\vec{b}_*\| + \cdots + \|\hat{A}_*\|^k \|\vec{b}_*\| \\ &= \|\vec{b}_*\| \frac{1 - \|\hat{A}_*\|^{k+1}}{1 - \|\hat{A}_*\|} \end{aligned} \quad (18)$$

where

$$\begin{aligned} \|\vec{b}_*\| &= \max_{u_k, v_k, \omega_k, \delta_k, T_s} \|\vec{b}_k\| \\ \|\hat{A}_*\| &= \max_{u_k, T_s} \|\hat{A}_k\| \end{aligned} \quad (19)$$

Since the state variables u_k, v_k, ω_k are all bounded by their physical meanings, as continuous functions defined over them, $\|\vec{b}_k\|$ and $\|\hat{A}_k\|$ must have maximum values accordingly. Therefore, given that Condition 1 is established, $\|\vec{b}_{k,0}\|$ is always bounded by $\frac{\|\vec{b}_*\|}{1 - \|\hat{A}_*\|}$ as k increases.

Remark 2: The proof is based on the linear approximation of the initial nonlinear difference model. Nevertheless, according to *Lagrange's mean value theorem*, there always exists $\tau \in [0, 1]$, satisfying

$$\begin{aligned} \vec{\varepsilon}_{k+1} &= A_{\text{mid}} \vec{\varepsilon}_k \\ A_{\text{mid}} &= J_{F,X}(X_k + (1 - \tau)X_{k+1}, U_k) \end{aligned} \quad (20)$$

And obviously, substituting (20) for (11) will not change the subsequent analysis and conclusion.

Remark 3: Here we give a sufficient instead of sufficient & necessary condition, which means that the violation of Condition 1 does not necessarily bring numerical instability. The main contribution of this condition, is the easy-to-satisfy property (see Fig. 2) under general settings and common speed, which imposes great practical meaning to the application of dynamic bicycle model (8).

V. SIMULATION

As pointed out by Jason Kong *et al.* (2015), a kinematic model has two main advantages: low-speed feasibility and computational efficiency [7]. Meanwhile, they also observe significant growth of error on the higher speed sinusoidal and winding track tests with kinematic model. Philip Polack *et al.* further find that $a_y \leq 0.5\mu g$ is decisive (2017), otherwise error of the kinematic bicycle model will become very large [2].

Therefore, it is necessary to verify the low-speed feasibility of the proposed model, and compare the differences between both models. Additionally, it is also necessary to evaluate the error of our model resulting from the modeling approximation and simplifications.

In what follows, we choose a vehicle (C-Class, Hatchback 2017) in *CarSim* as the prototype. *CarSim* is a multibody dynamics simulation software with high fidelity, modeling nonlinearity of tire force, steering system, suspension system and other components. All the parameters are collected (e.g. combining the two front wheels means doubling the vertical load to the equivalent single wheel and then determine the cornering stiffness by linearization) and listed in Table I. Open-loop simulation is about step steer response, whereas the closed-loop simulation involves a nonlinear MPC controller undertaking a stop-and-go task.

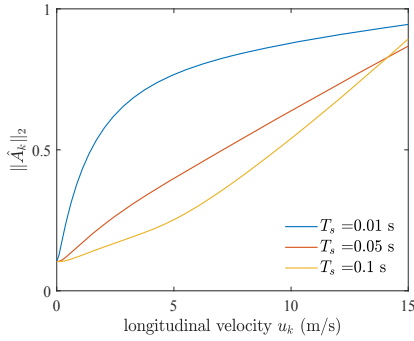
A. Open-loop Control

Data and codes of open-loop control are available at <https://github.com/gqgit/IEEE-IV2021-numerically-stable-dynamic-bicycle-model>.

TABLE I. Simulation Parameters

Parameter	Description	Value
I_z	yaw inertia of vehicle body	1536.7 kg·m ²
k_f	front axle equivalent sideslip stiffness	-128916 N/rad
k_r	rear axle equivalent sideslip stiffness	-85944 N/rad
l_f	distance between C.G. and front axle	1.06 m
l_r	distance between C.G. and rear axle	1.85 m
m	mass of the vehicle	1412 kg
N_p	predictive horizon	20
N_c	control horizon	1

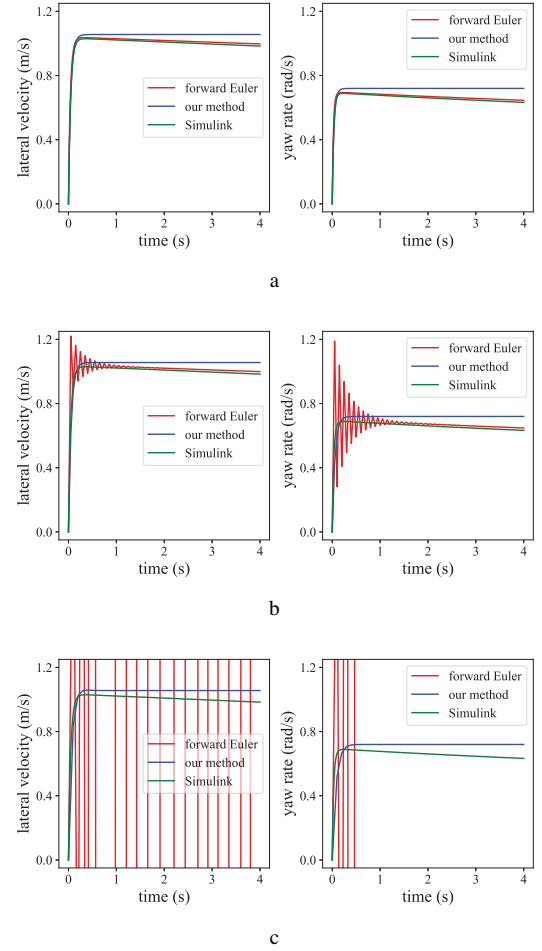
1) *versus Forward-Euler discretization*: The numerical stability, as well as the insensitivity regarding step length and driving speed, is rather important for autonomous driving applications. Before comparison, we first verify if the numerical stability Condition 1 is satisfied. Let $0 \text{ m/s} \leq u_k \leq 15 \text{ m/s}$, the 2-norm of $\hat{A}_k$ is shown in Fig. 2. Since $\|\hat{A}_k\|_2 \leq 1$, the numerical stability of our model is essentially guaranteed.

Fig. 2. Range of $\|\hat{A}_k\|_2$ defined over varying u_k

The model (1) with assumptions (3a and 3b) is built in *MATLAB/Simulink* with 4-order Runge-Kutta solver with a step length of 0.001s. The proposed model shows expected stability over time, and consistency over varying discretization step length T_s (Fig. 3). In contrast, the forward Euler method cannot hold this feature as T_s increases.

2) *versus Kinematic Model*: The open-loop step response of dynamic model (discretized by our method) and kinematic model (discretized by forward Euler method) are calculated, at different velocities. Simultaneously, the same input is imposed on the *CarSim* prototype (Adams-Moulton second-order method, $T_s = 0.001\text{s}$) to approximate experimental values in a field test.

The open-loop trajectories of both models and their error in respect to *CarSim* are shown in Table II. There is significant error reduction compared to the kinematic model, indicating that the model could improve the control accuracy of model-based controllers. Specifically, Fig. 4(a) reveals that the trajectory error mainly originates from longitudinal modeling, where we suppose $v\omega - \frac{1}{m}F_{Y1} \sin \delta \approx 0$. To evaluate the impact from $\cos \delta \approx 1$, one may substitute $k_f \cos \delta$ for all k_f in (8). Note that this will slightly change value of $\|\hat{A}_k\|$ but hardly reverse the numerical stability. The trajectory error will be even smaller, e.g. 3.05 m at $u_0 = 10 \text{ m/s}$ compared to 3.22 m in table II.

Fig. 3. Lateral state with varying discretization step length under step steering input. Conditions: $u_0 = 8 \text{ m/s}$, $\delta = 0.2674 \text{ rad}$
(a) $T_s = 0.01 \text{ s}$, (b) $T_s = 0.05 \text{ s}$, (c) $T_s = 0.1 \text{ s}$ TABLE II. Location RMS Error of 4-second Open-Loop Trajectory under Step Steer Input
Prototype: *CarSim* C-Class, Hatchback 2017

u_0 (m/s)	our model (m)	forward kinematic (m)	Accuracy Improvement
1	0.31	0.32	+2%
2	0.42	0.45	+7%
3	0.39	0.46	+16%
4	0.58	0.86	+33%
5	0.65	1.28	+49%
6	0.76	1.91	+60%
7	0.98	2.77	+65%
8	1.39	3.88	+64%
9	2.06	5.28	+61%
10	3.22	7.18	+55%

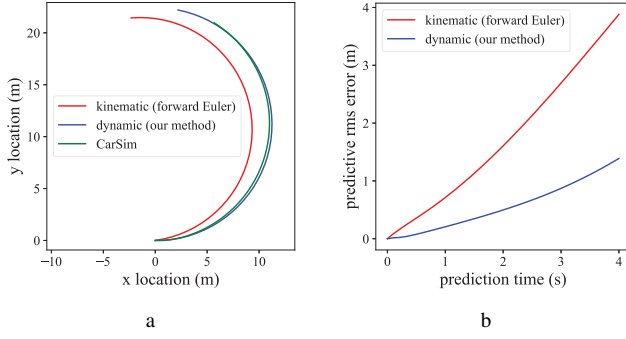


Fig. 4. Open-loop trajectory and error under step steer input. Conditions: $u_0 = 8$ m/s, $\delta = 0.2674$ rad (a) trajectory, (b) location error

B. Closed-loop Control

The open-loop control test has verified modeling error by constant steering angle and zero acceleration. In a predictive controller, the receding-horizon self-correction will help compensate for the model error dynamically. We design a nonlinear model predictive controller with (8) to comprehensively verify its effect in practical application. Starting from (0, 0), the vehicle model is always tracking a direct path connecting its center of gravity (C.G.) and the target (30, 30). The reference speed is set to be 6 m/s. There is an obstacle (light blue round area in Fig. 5) located at (15, 15) blocking its way. The parameters are deliberately co-designed so that the vehicle can not bypass with steering, but only stop to avoid collision. Note that collision is defined by overlapping of the obstacle and C.G. of the vehicle, instead of vehicle body profile. Once the vehicle stops, the obstacle is moved to (18, 12) (dark blue round area in Fig. 5), giving way to the vehicle. Then the vehicle starts to execute a stop-and-go task. To be emphasized, obstacle avoidance requires the vehicle to steer and accelerate simultaneously, which is similar to a left-turning maneuver after red light at intersections.

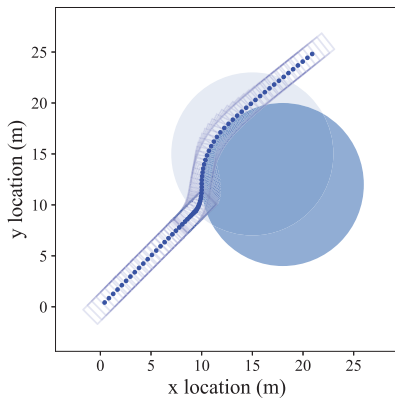


Fig. 5. Trajectory of the vehicle, avoiding a switching obstacle

The longitudinal velocity is shown in Fig. 6, whereas the longitudinal input a and lateral input δ are recorded in Fig. 7(a) and (b) respectively. The constrained nonlinear optimal

control problem is formulated as (21), satisfying (22).

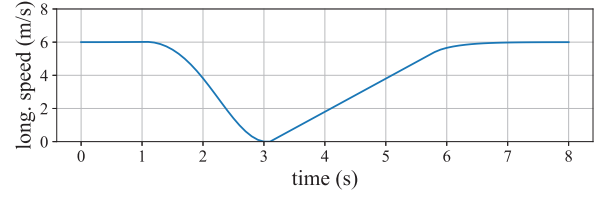
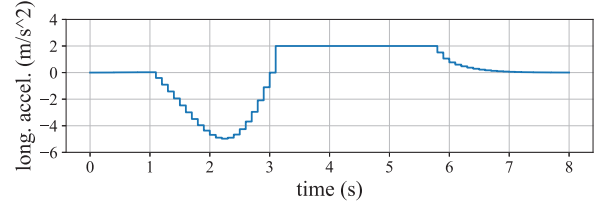
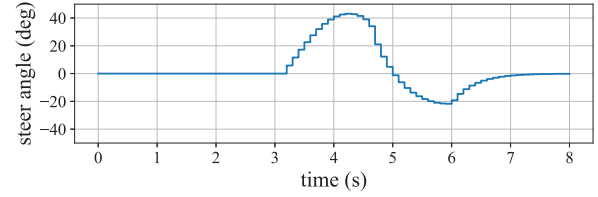


Fig. 6. Longitudinal velocity of the vehicle



a



b

Fig. 7. MPC control input (a) longitudinal input, (b) lateral input

$$\min \sum_{k=0}^N (X(k) - X_R(k))^T Q (X(k) - X_R(k)) + U(k)^T R U(k) \quad (21)$$

$$X(k+1) = F(X(k), U(k)) \quad (22a)$$

$$(X(k) - X_{\text{obs}}(k))^T Q_s (X(k) - X_{\text{obs}}(k)) \geq D_s^2 \quad (22b)$$

$$X_{\min} \leq X(k) \leq X_{\max} \quad (22c)$$

$$U_{\min} \leq U(k) \leq U_{\max} \quad (22d)$$

$X_R(k)$ is the k th reference trajectory point in the predictive horizon. $X_{\text{obs}}(k)$ conveys location of the obstacle at k th step in the predictive horizon. $Q = \text{diag}(100, 100, 0, 0, 0, 0)$, $R = \text{diag}(10, 500)$, $Q_s = \text{diag}(1, 1, 0, 0, 0, 0)$, $D_s = 8$. $X_{\min} = [-\infty, -\infty, -\infty, 0, -4, -3]$, $X_{\max} = [+\infty, +\infty, +\infty, 20, 4, 3]$. $U_{\min} = [-5, -\pi/4]$, $U_{\max} = [2, \pi/4]$. All quantities are in the international system of units. The discretization step length $T_s = 0.1$ s. One may also adjust value of T_s , trading off between prediction accuracy and computation burden.

It is important to point out that kinematic model is the only choice to be embedded into the predictive controller to carry out this mission, before our model is introduced. An attempt with forward-Euler discretized dynamic model will

fail the solver as u approaches zero, which has been verified by us in simulation.

Our model is computationally efficient as well. The average computation time of one-step MPC solution is 56.2 ms, whereas the same simulation carried out with kinematic model (4) has an average of 55.1 ms. The problem is solved by the nonlinear OCP solver *ipopt* [15] under the framework *CasADi* [16], on a laptop with an *intel core i7* CPU.

VI. CONCLUSION

In this paper, we propose a discretized dynamic bicycle model inspired by backward Euler method and a sufficient condition that guarantees its numerical stability. The condition turns out easy to achieve under general vehicle parameters and common urban driving speed (e.g. $\leq 15\text{m/s}$). Our method outperforms forward-Euler discretized dynamic model in terms of stability and shows an advantage in accuracy up to 65% compared to the kinematic model. A comprehensive stop-and-go task further verifies its effectiveness. It is a state transition model suitable for predictive controller design, model-based reinforcement algorithm training, etc., under general urban driving scenarios.

ACKNOWLEDGMENT

The authors are grateful to Hao Chen and Xuewu Lin for their valuable advice in matrix product derivation. We also appreciate the help from Guojian Zhan and Yuxuan Jiang in model verification. This work was presented at The 2nd international workshop on Chassis Systems Control and Autonomous Driving control (CSC-AD'21) (WS08), IV2021.

REFERENCES

- [1] S. Antonov, A. Fehn, and A. Kugi, "Unscented kalman filter for vehicle state estimation," *Vehicle System Dynamics*, vol. 49, no. 9, pp. 1497–1520, 2011.
- [2] P. Polack, F. Althché, B. d'Andréa Novel, and A. de La Fortelle, "The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?" in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 812–818.
- [3] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Transactions on control systems technology*, vol. 15, no. 3, pp. 566–580, 2007.
- [4] Z. Lin, J. Duan, S. E. Li, H. Ma, Y. Yin, and B. Cheng, "Continuous-time finite-horizon adp for automated vehicle controller design with high efficiency," in *2020 3rd International Conference on Unmanned Systems (ICUS)*. IEEE, 2020, pp. 978–984.
- [5] R. Rajamani, *Vehicle dynamics and control*. Springer Science & Business Media, 2011.
- [6] Y. Zheng, S. E. Li, J. Wang, D. Cao, and K. Li, "Stability and scalability of homogeneous vehicular platoon: Study on the influence of information flow topologies," *IEEE Transactions on intelligent transportation systems*, vol. 17, no. 1, pp. 14–26, 2015.
- [7] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *2015 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2015, pp. 1094–1099.
- [8] D. Xu, Y. Shi, and Z. Ji, "Model-free adaptive discrete-time integral sliding-mode-constrained-control for autonomous 4wmv parking systems," *IEEE Transactions on Industrial Electronics*, vol. 65, no. 1, pp. 834–843, 2017.
- [9] B. Li, Y. Zhang, Y. Zhang, N. Jia, and Y. Ge, "Near-optimal online motion planning of connected and automated vehicles at a signal-free and lane-free intersection," in *2018 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2018, pp. 1432–1437.
- [10] M. Cai, Q. Xu, K. Li, and J. Wang, "Multi-lane formation assignment and control for connected vehicles," in *2019 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2019, pp. 1968–1973.
- [11] J. C. Butcher and N. Goodwin, *Numerical methods for ordinary differential equations*. Wiley Online Library, 2008, vol. 2.
- [12] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of machine learning research*, vol. 18, 2018.
- [13] A. Nagariya and S. Saripalli, "An iterative lqr controller for off-road and on-road vehicles using a neural network dynamics model," in *2020 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2020, pp. 1740–1745.
- [14] R. Jungers, *The joint spectral radius: theory and applications*. Springer Science & Business Media, 2009, vol. 385.
- [15] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [16] J. A. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "Casadi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.